



Human Slop In, AI Slop Out

*Preprocessing as
the Origin of Retrieval Quality in
Retrieval-Augmented Generation*

A part of the series:

Building RAG Before the Model Runs

Jesper Karlsson · Otrobonita AI Labs

1/11

1. Opening

Preprocessing as a hallucination preventer

A retrieval-augmented generation (RAG) system exists to anchor a model's answer in supplied evidence — and a grounded answer can only be as reliable as the evidence it rests on. When the retrieved text is corrupted, ambiguous, or stripped of the structure that made it meaningful, the model is not merely under-informed; it is actively misinformed, and it has no way to know. Two things can then happen, and it helps to separate them. The model may *faithfully* report the corrupted evidence — a **grounding error**, whose cause is entirely upstream — or, handed context with a hole in it, it may do what language models do and fill the hole with something plausible — a **generation error**. This paper is about the first, and about how often the second is really the first in disguise: failures we diagnose at the generation layer whose causal origin was set before the model ever ran.

Preprocessing is the stage where that damage is either prevented or locked in. It is the work of turning a raw source into clean, structured text an index can trust — and because it happens first, it is the earliest and cheapest place in the entire pipeline to stop a hallucination before it can occur. Every later safeguard, from careful chunking to reranking to prompt design, is working to protect meaning that preprocessing either preserved or destroyed. This paper's argument is that preprocessing deserves to be treated as a correctness stage in its own right, not as janitorial work that precedes the real system.

The claim is easiest to feel with a concrete source. Pull up the declassified *Apollo 7 Onboard Voice Transcription* and the striking thing is how little looks wrong. You might expect a degraded 1968 document to be strewn with *[GARBLE]* placeholders and bracketed guesses. It is not: search the whole transcript for "garble" and you find only a handful of hits, and every one is a crew member or CapCom *saying* it — "you were a little bit garbled, but I think we've got most of it" — not a transcriber flagging lost audio. The pages read cleanly. That is exactly what makes them dangerous. The damage is not loud and self-announcing; it is quiet and structural — a typed "4" open enough at the top that OCR reads the timestamp ...04 as ...0h, and three columns of time, speaker, and speech held together by nothing but their position on the page.

The people who produced these documents in the 1960s did extraordinary work under impossible conditions. But they were transcribing for human historians, who read charitably and fill gaps from context without noticing they are doing it. They were not transcribing for a vector database, which reads literally and forgives nothing.

Feed that raw text straight into a RAG system and nothing warns it that a timestamp has been corrupted or that two speakers have been braided into a single line. The error is indexed with full confidence precisely because it does not look like an error. When a query later lands on that passage, the model can only work with what extraction left behind: it may report the corrupted value verbatim, or paper over the gap with a plausible guess. Either way, the failure that surfaces at answer time was largely authored back here, at extraction — a grounding error wearing a generation error's clothes.

This is worth saying plainly, because it inverts a common complaint — and it is meant, in the spirit of this paper's title, as a deliberate provocation:

Much of what we now dismiss as "AI slop" begins life as human slop — upstream, in the source, long before a model is involved. The model did not invent the mess. It inherited it, and amplified it.

That reframing is the subject of this paper. The central claim, stated precisely:

Preprocessing is not janitorial work that precedes the real system. It is the stage at which the information structure a retrieval system depends on is either created or destroyed. The topology that chunking later aligns to — and that faithful generation ultimately rests on — is born here, or it is lost here.

2. Two upstream origins of hallucination

Hallucination is usually discussed as a generation-time failure: the model, at the final step, says something untrue. Often the cause is set much earlier. Upstream of the model sits a whole chain of places where an answer can be spoiled — source selection, extraction, metadata, embedding, retrieval, ranking, context assembly — and a complete account would treat all of them. This series does not attempt that. It isolates the two upstream mechanisms it can say the most useful things about: the one this paper owns, and the one its sequel owns.

The preprocessing origin — meaning corrupted. When source text is garbled, ambiguous, or stripped of the marks that disambiguate it, the retrieval system indexes noise as if it were signal. A silently mis-read timestamp is retrieved with the same confidence as a correct one. A speaker label lost during extraction turns a two-person exchange into a single incoherent monologue. The model is then grounded on context that is quietly wrong, and it cannot know that it is. This is the origin this paper claims as its own.

The chunking origin — meaning fragmented. Even from clean source text, a poorly chosen chunk boundary severs a fact from the context that gives it meaning. A pronoun is stranded from its referent; a number is cut off from its units; a punchline is separated from its setup. Chunks that are too short, or that carry too little overlap, produce retrieval units that are individually fluent and collectively misleading. This origin is the subject of the companion paper and is treated there in full.

These are not the same failure, but they rhyme: in both, the model is handed context that is missing something, and the damage was done before it ran. **Preprocessing loses meaning by corrupting or decontextualising the text; chunking loses meaning by fragmenting it.** This paper owns the first; it points forward to the second.

3. Where this paper sits in the series

This paper sits in the middle of the pipeline, and it is worth being exact about the order. A companion paper on **data collection** genuinely comes first: sourcing and acquisition — deciding what is worth ingesting and pulling it in, *and choosing the frame on reality the system will represent* — is a distinct problem, and it precedes everything discussed here. This paper assumes collection has already happened, a raw source is in hand, *and a frame has been chosen*. The **chunking** paper is the sequel: it assumes the work described here is already done. So preprocessing does not come *first* — it comes *after* collection and *before* chunking.

What that middle position turns on is a single dependency. The chunking paper assumes that structural signals — section boundaries, speaker attributions, timestamps, document hierarchy — already exist to be aligned to. It has to assume this; alignment is meaningless without something to align.

But those signals are not found in raw sources. They are *manufactured* during preprocessing: recovered from inconsistent formatting, reconstructed where the original lost them, attached as metadata where they were only ever implicit. Structuring and enrichment is not the tail end of cleanup — it is where a document's information topology is first made explicit and machine-legible.

Preprocessing depends on sourcing's frame choice

Upstream sourcing — what reality the corpus represents, whether it stands alone or routes through a mesh — is not merely a data-collection phase. It is the frame within which preprocessing must operate. If sourcing chose a tight frame (Apollo with GET timestamps and speaker codes; Twain with chapters and letters), preprocessing must preserve those signals. If sourcing chose a partial, meta-level frame (knowledge *about* RAG, not complete RAG), preprocessing must still maintain what coherence exists, because later stages depend on it.

The topology that preprocessing manufactures, layer by layer, is meaningful *only within the frame that sourcing established*. Get the frame wrong upstream, and all of preprocessing's work becomes labor in service of a false premise. Sourcing decides *what angle we're representing and whether we stand alone or route through a mesh*; preprocessing decides *whether the model can read it within that frame*.

4. What preprocessing actually is

Before the examples, a plain definition — because "preprocessing" is often used loosely to mean any dull work that happens before the interesting part.

Preprocessing is everything that turns an acquired raw source into a clean, structured document a retrieval system can index. It sits between *ingestion* (getting the source into the system at all) and *chunking* (splitting it into retrieval units), and is best understood as four steps — listed in sequence, though in practice they loop and inform one another:

- **Extraction** — lifting the actual content out of its container: text out of a PDF, HTML, or audio transcript.
- **Cleaning** — removing what is not content: boilerplate, navigation, headers and footers, OCR noise.
- **Normalisation** — making the surviving text consistent: encoding, Unicode, whitespace, casing.
- **Structuring & enrichment** — attaching the marks that carry meaning: section boundaries, speaker labels, timestamps. These are the topology signals every later stage depends on.

Two things follow, and they set up the rest of the paper. First, preprocessing is not one action but a short pipeline of its own, whose steps loop rather than merely follow — how you extract determines what there is to clean, and what you intend to structure determines how you must extract — so a fault in any step propagates into all of them. Second — the claim this paper defends — structuring and enrichment is not the last and dullest step but the *objective* the first three exist to serve: **it is where a document's information topology is first made explicit and machine-legible**. That is the point at which meaning is either recovered or lost for good, which is exactly what the two examples below are chosen to show.

5. Two worked examples

The argument is easiest to see at the extremes. This section places two real corpora at opposite ends of the preprocessing difficulty scale. One is nearly trivial to prepare; the other is among the hardest text a pipeline will meet. Neither is a contrived edge case — both are corpora this project actually ingests.

5.1 The hard end: the Apollo 7 onboard voice transcription

Our source is the declassified *Apollo 7 Onboard Voice Transcription* (Manned Spacecraft Center, Houston, December 1968). Two pages from it stand in for two different kinds of preprocessing damage.

Exhibit A — a transcript page (Day 1, p. 51). The instructive thing about this page is how *legible* it looks. There is no wall of unreadable garble; a human skims it without effort. The damage is quieter, and precisely the kind an eye forgives and an index does not. Read as five stacked layers, bottom to top:

- **Physical layer.** A fifty-year-old reproduction: speckle and dust across the page, faint skew, a struck-through *CONFIDENTIAL* header, and a black redaction bar over the footer. Before any character is read, the page is already carrying marks that mean nothing to the conversation.

- **OCR layer.** The typed "4" is open enough at the top that recognition reads it as an "h" — and the corruption lands squarely on the ground-elapsed-time values a retrieval system would key on:

On the page	OCR reads	Effect
00 02 12 04	00 02 12 0h	timestamp now unmatchable
00 02 12 43	00 02 12 h3	ordering broken
00 02 13 14	00 02 13 1h	event silently misfiled

High confidence, wrong output, no error raised. A query for the utterance at 12:04 will never find it, because in the index it is filed under a time that does not exist.

- **Layout layer.** The page is three columns held together only by visual position: GET timestamp · speaker code · utterance. A standard PDF text extractor reads *horizontally*, straight across the whole page, so the *timestamp | speaker | utterance* tuple is destroyed at the moment of extraction — *00 01 58 38 · LMP · SPS propellant tank temperature is running 70...* collapses into one undifferentiated line. A layout-aware or vision model can preserve the columns; a naïve extractor cannot, and most default pipelines use the naïve one. Multi-line utterances smear the braid further down the page.
- **Notation layer.** The domain conventions carry the meaning: speaker codes (LMP, CDR, CMP, and CC for CapCom/Houston), the *DD HH MM SS* time format, and the "..." that marks a clipped or overlapping transmission — not literal words, but a naïve pipeline ingests them as if they were.
- **Meaning layer.** Here is the subtlety that makes this corpus such a good teacher. The facts the RAG system wants — *who said what, when* — **do** exist explicitly on the page; they sit right there in their columns. What is only *implicit* is the **relationship** binding them: that this timestamp goes with that speaker and that utterance. The information is present; it is encoded **spatially rather than semantically**. Preprocessing's job is to convert that spatial encoding into an explicit one before the layers below dissolve it — which is the whole thesis in miniature.

Exhibit B — the cover page. The title page is the opposite failure mode: perhaps a dozen words of real content sitting in a field of visual noise. Each artifact is a distinct extraction hazard — OCR attempts the halftone dot-screen as characters, the FOIA *NOTICE* box rotated some eighty degrees reads as a diagonal smear of broken words, the struck-through classification stamp yields fragments, and handwritten declassification annotations scrawl across the header. A human reads *Apollo 7 Onboard Voice Transcription, December 1968* in a second; a naïve text pipeline sees mostly artifacts around a few real words and, left alone, indexes the noise as though it were content.

The point lands before any taxonomy does: *what looks like one text file is five stacked, leaky layers* — and on some pages the content-to-noise ratio approaches zero. Disciplined preprocessing here is not cleanup; it is reconstruction, layer by layer, of information the source no longer cleanly contains.

Exemplifier — the Space Talks Timewheel. This exact loss-and-recovery is visible, live, at spacetalks.otrobonita.com. Its Timeline Dial holds a mission moment fixed and lets you step one Apollo 11 exchange through three representations. Raw OCR shows the damage untouched — "Hello, Houston; Apollo II." (the "11" read as "II"), "Apollo l1, Apollo 1], this is Houstor through Tananarive," and a line that has decayed to pure noise: "k n--k i'ri A · - iMrllJ_l_ i i_--." Clean v2 shows the same lines repaired and tagged ✓ Healed v2 — the call sign restored to 11, the speaker turn intact. Two things make it an honest exhibit rather than a demo: the healing is labelled, so you can see which representation you are reading, and the worst-decayed fragment is left visibly ungarbled rather than silently invented — a reconstruction the pipeline declines to fake. (An annotated still overlay of p. 51 remains possible but is now optional; the live Timewheel does the work.)

5.2 The easy end: Project Gutenberg (Twain)

The same pipeline meets the *Adventures of Huckleberry Finn* Gutenberg edition and has almost nothing to do. Two things make it easy, and both are visible in the source.

First, the boilerplate is *bounded and labelled*. The file opens with the standard Project Gutenberg header — title line, license paragraph, and metadata fields (Author: Mark Twain; Illustrator: E. W. Kemble; release and update dates; credits) — and the body begins at a literal, machine-findable marker: *** START OF THE PROJECT GUTENBERG EBOOK ADVENTURES OF HUCKLEBERRY FINN ***, closed by its matching *** END ***. Stripping the wrapper is a single stable rule, not a judgement call.

Second, the structure is *already in the markup*. Inspect the rendered chapter and the text sits inside semantic HTML — clean UTF-8 prose in a <p> element, chapters under headings — so the paragraph and chapter topology the chunking layer will want is present and legible without reconstruction. There is no OCR layer, no braided columns, no notation to decode. The only real decisions are cosmetic: whether to keep the interleaved illustrations, and how to handle curly quotes and the em-dash in "Aunt Polly—Tom's Aunt Polly."

5.3 What the pair proves

Set side by side, the two corpora make the paper's structural claim concrete:

Preprocessing is corpus-dependent. The effort that rescues an Apollo transcript — column de-braiding, notation parsing, speaker-turn reconstruction — would be wasted labour on a clean Twain file, and the light boilerplate strip that perfectly serves Gutenberg would leave the Apollo scans in ruins.

This is exactly the argument of the companion chunking paper, moved one stage upstream. There is no universal chunking strategy because information topology varies by corpus; there is no universal *preprocessing* strategy because the **legibility** of that topology varies by corpus first.

The deeper way to state it — and the spine of this whole series — is this:

Preprocessing is the transformation of a document's information topology from a human-readable encoding into a machine-readable one.

Seen that way, the two exhibits differ only in how far apart those two encodings sit. Gutenberg's topology is already *semantic*: a <p> **is** a paragraph, a heading **is** a chapter boundary, so the distance is nearly zero and preprocessing has almost nothing to do. Apollo's topology is merely *spatial*: a column position that a human eye resolves instantly and a naïve extractor destroys, so the distance is large and preprocessing must do real reconstruction. The preprocessing burden of any corpus is simply that distance. (The next section adds a third encoding — topology carried *organizationally*, in whom a claim is true for — which is invisible on the page altogether.) One corpus shows the burden can be near zero; the other shows it can be the largest line item in the build; most real corpora sit on the line between them.

How grounding errors trace back to sourcing

When a sourcing decision produces a frame that preprocessing cannot fully honor, grounding errors result. If sourcing chose a frame that requires timestamps (Apollo), but the original source is too degraded to recover timestamps reliably, preprocessing cannot fix it. It can only faithfully report the corruption (a grounding error) or invent around it (a generation error wearing preprocessing's clothes). Every failure mode in preprocessing's taxonomy traces back to either corrupted source *or* a frame choice that didn't account for what the source actually contains. This is why the first paper is about frame honesty: sourcing must know whether the frame it chose is defensible given what the source permits.

6. When clean text still lies: contradiction and lost scope

The Apollo case is loss by *corruption* — the words themselves were damaged. There is a second, quieter failure that befalls text which is perfectly clean, and stakeholders meet it far more often: loss by *decontextualization*.

Consider a large organisation whose departments use the same system differently. Operations runs seven processes on System X; engineering runs twelve. Both statements are true — each within its own department. Ingest both into one flat corpus and the boundary that made them compatible is gone. What the retrieval layer now holds is a bare contradiction: *System X supports 7 processes* and *System X supports 12 processes*, side by side, with nothing to say which belongs to whom.

Note what did **not** go wrong. Neither source was inaccurate; neither was garbled. The failure was entirely in preprocessing — the scope, *which department this claim holds for*, was never captured as a field, so two claims that were never really in conflict were filed as though they were. (The System

X figures are illustrative; the pattern is not — it recurs wherever one system serves several organisational units.)

Many apparent contradictions in an enterprise RAG corpus are not contradictions at all. They are claims whose scope or version metadata was lost in preprocessing.

Genuine contradictions do exist — two sources can simply disagree, a measurement can be wrong, a policy can be disputed — and those need editorial resolution, not a metadata fix. But a large share of what surfaces as contradiction in enterprise corpora is nothing of the kind, and it comes in two flavours, each with a different missing field:

- **Lateral (scope).** Department, region, product variant, customer tier. Both claims are true at once; they belong to different boundaries. Missing field: *scope*.
- **Temporal (version).** Revision 1 of an instruction versus revision 3, where the later supersedes the earlier and only the current one applies unless history is explicitly requested. Missing field: *effective date / revision*.

The consequences are worse than steady wrongness, for two reasons. First, the system fails **inconsistently**: whichever chunk happens to rank higher wins, so the same question returns "7" today and "12" tomorrow. Inconsistent wrongness erodes trust faster than reliable wrongness, because a user cannot even learn where to be careful. Second — and this is the mechanism from §1 again — handed two flattened claims, the model may **invent a reconciliation**: "*System X supports 7 core and 5 optional processes,*" a confident figure true in no department at all. The gap it fills is the missing scope; the fill is a fabrication that sounds more authoritative than either real answer.

The fix lands exactly where this paper already points — the **structure & enrich** step. Preprocessing must attach the boundary (scope: engineering, effective_date: 2025-03) so the retrieval layer can either filter to the asker's context or, better, return both claims *with their scopes intact*: "For operations, seven; for engineering, twelve." Stated that way it is not a contradiction at all — it is a complete, correct answer. Retrieval does the filtering, but it can only filter on metadata that preprocessing created. Preprocessing therefore creates not only structural topology but **scope topology**: the boundaries within which each claim is true.

This forces an honest widening of the definition, and the paper embraces it rather than retreating from it:

In RAG, preprocessing ends not when the text is clean, but when the boundaries required to interpret that text — structural, temporal, and organisational — are machine-readable.

That is more than "clean the document." It is the beginning of knowledge modelling — and it is the natural terminus of the topology thesis: extraction recovers *structural* topology, and enrichment must go on to make *scope* topology explicit too, because both are load-bearing for a correct answer.

This points at **one** preventive architecture, though not the only one. Instead of pouring every department's knowledge into a single store and relying on metadata filters to keep scopes apart at

query time, you can partition the corpus by knowledge boundary — system, process, part-family — so incompatible scopes never share a store to begin with. The two approaches trade off: partitioning simplifies within-store correctness but introduces routing, duplication, and cross-boundary-retrieval problems; a unified store with disciplined metadata avoids those but leans entirely on the scope tags preprocessing attached. Neither is free, and the right choice is corpus-specific. (Partitioning is the stance behind Otrobonita's *Rag of Rags* — minted specialists scoped by boundary, "not one RAG per bolt, and not one masterbrain" — offered here as one worked response, not a conclusion the argument forces.)

7. What it costs to skip this

Preprocessing has an awkward profile for anyone deciding where to spend effort: it is invisible when it works, and nearly invisible when it fails. A broken preprocessing step rarely throws an error. It ships a clean-looking answer that happens to be wrong — the failure mode most corrosive to trust, because nothing flags it and nothing crashes.

That is the whole severity argument in one line:

Preprocessing is the cheapest stage to get right and among the most expensive to get wrong, because every stage downstream inherits its output. A shortcut here is paid back many times over — in confident wrong answers, in eroded user trust, and in debugging that begins at the wrong end of the pipeline.

Made concrete: a single silently mis-read value — a torque spec, a dosage, the timestamp on a safety event — is nearly free to prevent at ingestion and very expensive to discover once it has surfaced, months later, inside an authoritative-sounding answer a user acted on. That asymmetry is the entire budget argument.

The one decision this paper asks of a stakeholder is not technical. It is to drop the assumption that preprocessing is a solved, one-size step you can simply "run." The two examples show why that assumption is expensive: the same pipeline that is almost free on a clean Gutenberg text has to perform real reconstruction on an Apollo scan. Preprocessing effort must be **budgeted per corpus, at ingestion time** — before a delivery date is promised, not discovered after.

A rough rule of thumb for triage:

- **Clean, born-digital, well-marked-up sources** (Gutenberg-like): preprocessing is a rounding error. Proceed.
- **Scanned, multi-column, notation-heavy, or OCR'd sources** (Apollo-like): preprocessing is likely the single largest and riskiest line item in the build — and the cheapest place in the entire system to prevent a hallucination.

Everything above is settled before a single chunk is cut. What preprocessing hands downstream — clean text with its structure made explicit — is exactly what the chunking paper assumes it already has. Get this stage wrong and downstream cleverness — chunking, reranking, prompting — cannot *reliably* reconstruct what was lost. It can sometimes paper over the gap, drawing on a redundant

passage or the model's own knowledge, but not dependably, and least of all in the cases that matter most.

Human slop in, AI slop out. The corollary is the hopeful one: clean, well-structured, correctly-scoped input is the cheapest reliability a RAG system will ever buy.

About This Paper

Disagreement is the point. If part of this is wrong — and some of it will be — the author would rather hear it than not.

Cite as Karlsson, J. (2026). Human Slop In, AI Slop Out. Otrobonita AI Labs.
otrobonita.com/whitepapers

License Creative Commons Attribution 4.0 (CC BY 4.0). Share it, quote it, argue with it — attribution is all that is asked.

The authoring team

- **Jesper Karlsson** — concept, thesis, ideas, structure, and accountability for every final call.
- **Fable 5, GPT 4.5, Grok 4.5, and Gemini 3.6** — drafting support, fact-checking, handling of uncertainty and counter-evidence, independent critique, and adversarial review.

Seven iterations and approximately thirty-two individual feedback loops.

Written in 2026.